REPORT 001: EPOCH 1: LEGACY ELECTRON ARCHIVE, RUST EGUI INITIALIZATION, YAAE/TWOBOWLS THEME ENGINE, AND HARDWARE DEVICE MODELS

2026-11-04 00:01:00 | Epoch 1 of 20: Rust egui scaffold, src/theme.rs (15 yaae themes + 4 twoBowls presets), src/devices.rs (nanoKONTROL2, Akai MIDI Mix, Nord Drum 2)

Operator: Gemini 4 Argon (Medium)

Prompt

```
Epoch 1 of 20: Read docs/NKEDITOR-RUST-PORT-SPEC.md and README.md in the current
↪  directory. Archive the legacy Electron/TypeScript files (src/, tests/,
↪  vite.config.ts, tsconfig.json, tailwind.config.js, postcss.config.js, index.html)
↪  into legacy-electron/ for reference, and initialize the new pure Rust egui
↪  desktop application at the repository root (Cargo.toml, src/) plus a Node.js
↪  companion SDK directory (js/ and root package.json configured with 'node
↪  --test'). In Cargo.toml, depend on egui = '0.29', eframe = '0.29', yaae-theme =
↪  path = '/home/renick/claude/yaae/crates/yaae-theme', features = ['egui', 'serde']
↪  , serde =  version = '1', features = ['derive'] , serde_json = '1', and rosc =
↪  '0.10'. Implement (1) src/theme.rs integrating all 15 yaae_theme::ThemePreference
↪  variants (Default, Light, Dark, SolarizedDark, Monokai, Dracula, Nord,
↪  GruvboxDark, LightGrayscale, DarkGrayscale, HighContrastBw, HighContrastColor,
↪  HighContrastLight, CreamLowContrast, MediumContrast), the 4 twoBowls background
↪  presets (White, Cream, Black, Dark Grey), CustomColorOverrides hex
↪  parsing/formatting, zero-rounding crisp borders (Rounding::ZERO), and UI scaling
↪  (50% to 200%); and (2) src/devices.rs modeling the complete Korg nanoKONTROL2 (8
↪  tracks of knob CC 16-23, slider CC 0-7, solo CC 32-39, mute CC 48-55, rec CC
↪  64-71 + 11 transport buttons + 28 LEDs), Akai MIDI Mix (8 tracks of 3 knobs, 1
↪  fader, mute/solo/rec_arm notes + master section + 24 LEDs), and Nord Drum 2
↪  Desktop target synth CC definitions. Always use CARGO_BUILD_JOBS=2, never launch
↪  a blocking GUI window or live audio, never inspect sibling directories in
↪  /home/renick/antigravity/, run cargo test and npm test, git commit all changes,
↪  and author and compile reports/report-001.md into reports/report-001.html (and
↪  .pdf) using /home/renick/claude/html-report-generator/build-report.sh --pdf, then
↪  commit.
```

TL;DR

Archived the legacy Electron/React/TypeScript application into `legacy-electron/` and initialized the pure Rust `egui = "0.29"` / `eframe = "0.29"` desktop crate at the repository root alongside a companion Node.js SDK in `js/`. Implemented `src/theme.rs` integrating all 15 `yaae_theme::ThemePreference` variants, the 4 `twoBowls` background presets (`White`, `Cream`, `Black`, `Dark Grey`), `CustomColorOverrides` hex parsing/formatting, `Rounding::ZERO` crisp borders, and 50% to 200% UI scaling. Implemented `src/devices.rs` modeling the complete Korg nanoKONTROL2 (51 controls, 28 LEDs), Akai MIDI Mix (60 controls, 24 LEDs), and Nord Drum 2 Desktop target synthesizer CC definitions (32 parameters across 6 pad channels and global FX channel 10), verified by 14 passing automated tests (`10` Rust unit tests + `4` Node.js SDK tests).

1. Plain-Language Overview & Epoch 1 Goals

nkEditor is a desktop application that sits between physical music controllers (boxes with physical knobs, sliders, and light-up buttons that plug into a computer via USB) and digital synthesizers or music software. When a musician turns a knob or presses a button on a hardware controller, the controller sends a **MIDI** (Musical Instrument Digital Interface) message: either a **CC** (Continuous Controller, a number from 0 to 127 representing a knob/fader position or button press) or a **Note** message (a musical note number from 0 to 127 with an on/off velocity). nkEditor intercepts those messages, remaps them to whatever target parameters the musician wants, and keeps the physical **LEDs** (Light-Emitting Diodes inside the controller's buttons) synchronized with the software state.

Previously, nkEditor3 was built on **Electron** (which bundles an entire Chromium web browser and Node.js runtime to display a user interface). In this 20-epoch port (nkEditor4), we are replacing Electron with a fast, low-latency, pure **Rust** desktop application built on egui (an immediate-mode graphical user interface library in Rust) while providing a lightweight **Node.js / Konduktiva JavaScript SDK** that communicates with the Rust app over **OSC** (Open Sound Control, a fast network protocol over UDP for musical instruments).

In **Epoch 1 of 20**, we accomplished four foundational tasks:

1. **Archived the Legacy Electron Codebase**: Moved all Electron/React/TypeScript sources (src/, tests/, vite.config.ts, tsconfig.json, tailwind.config.js, postcss.config.js, index.html) into legacy-electron/ so the git history and reference behavior stay intact without cluttering the new Rust root.
2. **Initialized the Pure Rust egui Crate & Node.js Companion SDK**: Created Cargo.toml at the repository root configured with egui = "0.29", eframe = "0.29", yaae-theme, serde, serde_json, and rosc = "0.10", alongside a clean root package.json ("test": "node --test js/*.test.js") and js/nkeditor-sdk.js.
3. **Implemented the Visual Theme Engine (src/theme.rs)**: Connected all 15 named visual palettes from yaae_theme::ThemePreference, the 4 twoBowls surface background presets (White, Cream, Black, Dark Grey), manual CSS hex color overrides (CustomColorOverrides), sharp rectangular borders (egui::Rounding::ZERO), and UI zoom scaling (50% to 200%).
4. **Implemented Hardware Controller & Synthesizer Models (src/devices.rs)**: Modeled every physical knob, slider, fader, button, and LED on the **Korg nanoKONTROL2** and **Akai MIDI Mix**, plus the complete MIDI CC specification for the **Nord Drum 2 Desktop** percussion synthesizer.

Plain-English Concepts Mapped to Code Components

Plain-English Concept	What It Does for the Musician	Exact Code Component
Legacy Electron Archive	Preserves the original TypeScript/React app for reference	legacy-electron/src/, legacy-electron/tests/
Rust Desktop Application Root	Compiles the native Linux desktop binary and library	Cargo.toml, src/lib.rs, src/main.rs
15 Named Visual Themes	Lets the user switch between dark, light, and high-contrast studio palettes	src/theme.rs (yaae_theme::ThemePreference, ThemeSettings::apply_to_ctx)
4 twoBowls Background Presets	Overrides the main canvas background (White, Cream, Black, Dark Grey)	src/theme.rs (TwoBowlsBackgroundPreset)

Plain-English Concept	What It Does for the Musician	Exact Code Component
Custom Hex Color Overrides & Crisp Borders	Parses `#RRGGBB` / `#RRGGBBAA` strings and enforces `Rounding::ZERO` sharp edges	`src/theme.rs` (`CustomColorOverrides`, `parse_hex_color`, `enforce_zero_rounding`)
UI Zoom Scaling (50% to 200%)	Scales the entire interface for compact laptops or high-DPI studio monitors	`src/theme.rs` (`clamp_ui_scale_percent`, `ThemeSettings::zoom_factor`)
Korg nanoKONTROL2 Hardware Model	Declares 8 tracks (knob, slider, solo/mute/rec), 11 transport buttons, and 28 LEDs	`src/devices.rs` (`nanokontrol2_definition`, `NK2_KNOB_CCS`, `NK2_ALL_LED_CCS`)
Akai MIDI Mix Hardware Model	Declares 8 tracks (3 knobs, 1 fader, 3 Note buttons), 4 master controls, and 24 LEDs	`src/devices.rs` (`akai_midimix_definition`, `MIDIMIX_KNOB_ROW1_CCS`)
Nord Drum 2 Synth CC Dictionary	Maps all 26 per-pad drum synth parameters (Ch 1 to 6) and 6 global FX parameters (Ch 10)	`src/devices.rs` (`nord_drum_2::tone`, `noise`, `click`, `mix`, `fx`, `channels`)
Companion Node.js SDK	Exposes matching constants and OSC endpoints to external Konduktiva scripts	`js/nkeditor-sdk.js`, `js/nkeditor-sdk.test.js`, `package.json`

2. Phase 1: Archiving Legacy Electron & Initializing the Rust/JS Workspace

Repository Restructuring and Crate Configuration

To transition cleanly from Electron to Rust without losing any reference code from `nkEditor3`, we used `git mv` to relocate `src/`, `tests/`, `vite.config.ts`, `tsconfig.json`, `tailwind.config.js`, `postcss.config.js`, and `index.html` into `legacy-electron/` (along with copies of the legacy `package.json` and `package-lock.json`).

At the repository root, we created `Cargo.toml` for `nkeditor` v4.0.0 with both a `[lib]` target (`src/lib.rs`) and a `[[bin]]` target (`src/main.rs`), depending on:

- `egui = "0.29"` and `eframe = "0.29"` for immediate-mode native GUI rendering.
- `yaae-theme = { path = "/home/renick/claude/yaae/crates/yaae-theme", features = ["egui", "serde"] }` for ecosystem-wide visual palettes and WCAG-compliant focus rings.
- `serde = { version = "1", features = ["derive"] }` and `serde_json = "1"` for configuration and preset serialization.
- `rosc = "0.10"` for Open Sound Control (OSC) packet encoding and decoding.

In `src/main.rs`, `NkEditorApp` initializes `ThemeSettings` and `get_all_device_definitions()`, and supports non-blocking CLI flags (`--version` and `--check-init`) so automated verification never spawns a blocking GUI window.

3. Phase 2: Visual Theme Engine (‘src/theme.rs‘)

Integrating ‘yaae-theme‘, ‘twoBowls‘ Presets, Hex Overrides, and Scaling

In everyday terms, musicians work in environments ranging from pitch-black live stages to bright daylight studios. `src/theme.rs` provides a unified theming layer that combines ecosystem-standard `yaae-theme` palettes with `twoBowls` surface presets and per-element hex overrides:

- **All 15 `yaae_theme::ThemePreference` Variants**: Supports `Default`, `Light`, `Dark`, `SolarizedDark`, `Monokai`, `Dracula`, `Nord`, `GruvboxDark`, `LightGrayscale`, `DarkGrayscale`, `HighContrastBw`, `HighContrastColor`, `HighContrastLight`, `CreamLowContrast`, and `MediumContrast`, with case-insensitive name parsing via `parse_theme_preference`.

- **4 `twoBowls` Background Presets (`TwoBowlsBackgroundPreset`)**: Supports `White` (`#FFFFFF`), `Cream` (`#F8F2E4`), `Black` (`#000000`), and `DarkGrey` (`#181818`).
- **Manual CSS Hex Overrides (`CustomColorOverrides`)**: Parses `#RGB`, `#RGBA`, `#RRGGBB`, and `#RRGGBBAA` strings via `parse_hex_color` and formats `egui::Color32` values via `format_hex_rgb`, `format_hex_rgba`, and `format_hex_color`. Supports custom overrides for `background`, `background_darker`, `surface`, `border`, `text`, `accent`, `solo`, `mute`, and `rec`, including a built-in `CustomColorOverrides::nkeditor3_classic()` preset matching the legacy hardware colors (`#FF6600` accent, `#FFCC00` solo, `#00CC66` mute, `#FF3333` rec).
- **Zero-Rounding Crisp Borders (`enforce_zero_rounding`)**: Sets `window_rounding`, `menu_rounding`, and all widget state roundings (`noninteractive`, `inactive`, `hovered`, `active`, `open`) to `egui::Rounding::ZERO`.
- **UI Scaling (`50%` to `200%`)**: `clamp_ui_scale_percent` clamps user percentages to `50..=200`, and `ThemeSettings::apply_to_ctx` applies the resulting `0.5..=2.0` zoom factor directly to `egui::Context`.

4. Phase 3: Hardware Controller & Synthesizer Models (‘src/devices.rs‘)

Korg nanoKONTROL2, Akai MIDI Mix, and Nord Drum 2 Definitions

Different MIDI controllers speak slightly different dialects of MIDI. For example, pressing a button on the Korg nanoKONTROL2 sends a **Control Change (CC)** message, whereas pressing a button on the Akai MIDI Mix sends a **Note On/Off** message. `src/devices.rs` captures these exact hardware layouts in pure data structures (`DeviceDefinition`, `TrackLayout`, `ContinuousControlDef`, `ButtonControlDef`, `ExtraControlsSection`, `LedProtocol`, `LedDef`):

Hardware Controller & Target Synthesizer Specifications in ‘src/devices.rs‘

Device / Synth	ID / USB ID	Tracks / Channels	Continuous Controls	Button Controls	LED Protocol & Count
Korg nanoKONTROL2	`nanokontrol2` (`0944:0117`)	8 tracks (Ch 1) + `transport` section	8 Knobs (`CC 16..23`), 8 Sliders (`CC 0..7`)	8 Solo (`CC 32..39`), 8 Mute (`CC 48..55`), 8 Rec (`CC 64..71`), 11 Transport (`CC 41..46, 58..62`)	CC on Ch 1 (`127` on, `0` off), **28 LEDs** (24 track + 4 transport: Play, Stop, Record, Cycle)
Akai MIDI Mix	`akai-midimix` (`09e8:0031`)	8 tracks (Ch 1) + `master` section	Row 1 Knobs (`CC 16,20,24,28,46,50,54,58`), Row 2 (`CC 17,21,25,29,47,51,55,59`), Row 3 (`CC 18,22,26,30,48,52,56,60`), 8 Faders (`CC 19,23,27,31,49,53,57,61`), Master Fader (`CC 62`)	8 Mute (`Note 1,4,7,10,13,16,19,22`), 8 Solo (`Note 2,5,8,11,14,17,20,23`), 8 Rec Arm (`Note 3,6,9,12,15,18,21,24`), Bank Left/Right (`Note 25,26`), Solo Master (`Note 27`)	Note on Ch 1 (`127` on, `0` off), **24 LEDs** (8 mute, 8 solo + 8 rec_arm)

Device / Synth	ID / USB ID	Tracks / Channels	Continuous Controls	Button Controls	LED Protocol & Count
Nord Drum 2 Desktop	`nord_drum_2` (Target Synth)	Pads 1..6 on `Ch 1..6`, Global FX on `Ch 10`	**Tone** (12 CCs: `30, 31, 46..55`), **Noise** (9 CCs: `14..22`), **Click** (2 CCs: `56, 57`), **Mix** (3 CCs: `7, 10, 58`), **Global FX** (6 CCs: `23..28`)	Triggered via Pad Note messages	Target synthesizer (`32` total CC parameters in `ALL_PARAMS`)

Tests

- `CARGO_BUILD_JOBS=2 cargo test` — 10 Rust unit tests passing (`all_15_yaae_themes_are_supported_and_four_twobowls_background_presets_match_specification`, `hex_parsing_and_formatting_roundtrip`, `custom_color_overrides_apply_to_visuals_and_palette`, `ui_scaling_clamps_between_50_and_200_p` `nanokontrol2_definition_matches_hardware_spec`, `akai_midimix_definition_matches_hardware_spe` `device_registry_and_port_identification_work`, `structural_consistency_no_duplicate_midi_assi` `nord_drum_2_cc_and_channel_constants_are_verified`), 0 failures
- `npm test` (`node --test js/*.test.js`) — 4 Node.js SDK unit tests passing, 0 failures

QA

```
`14 tests passing (10 Rust + 4 Node.js)`

`15 yaae-theme variants + 4 twoBowls presets verified`

`51 nanoKONTROL2 controls (28 LEDs) + 60 MIDI Mix controls (24 LEDs) + 32 Nord Drum 2 CCs verified`

`0 warnings, 0 failures`
```

All Epoch 1 deliverables compiled cleanly with `CARGO_BUILD_JOBS=2` and passed both `cargo test` and `npm test` without launching a blocking GUI window or live audio.

Files touched

- ‘Cargo.toml‘ — Initialized pure Rust crate with ‘egui = ”0.29”‘, ‘eframe = ”0.29”‘, ‘yaae-theme‘, ‘serde‘, ‘serde_json‘, and ‘rosc = ”0.10”‘
- ‘src/lib.rs‘ — Library root exporting ‘devices‘ and ‘theme‘ modules
- ‘src/main.rs‘ — Desktop binary entry point (‘NkEditorApp‘) with headless ‘--version‘ and ‘--check-init‘ flags
- ‘src/theme.rs‘ — Theme engine supporting all 15 ‘yaae_theme::ThemePreference‘ variants, 4 ‘twoBowls‘ background presets, ‘CustomColorOverrides‘ hex parser/formatter, ‘Rounding::ZERO‘ crisp borders, and 50%-200% UI scaling
- ‘src/devices.rs‘ — Complete hardware definitions for Korg nanoKONTROL2 (51 controls, 28 LEDs), Akai MIDI Mix (60 controls, 24 LEDs), device registry/port matching, and Nord Drum 2 Desktop CC & channel constants (32 parameters)
- ‘package.json‘ — Root Node.js package configured with ‘node --test js/*.test.js‘

- ‘js/nkeditor-sdk.js‘ — Companion Node.js / Konduktiva JavaScript SDK constants and OSC endpoint definitions
- ‘js/nkeditor-sdk.test.js‘ — Node.js built-in test runner suite verifying SDK parity with Rust definitions
- ‘.gitignore‘ — Added ‘target/‘ for Rust build artifacts
- ‘legacy-electron/‘ — Archived legacy Electron/React/TypeScript files (‘src/‘, ‘tests/‘, ‘vite.config.ts‘, ‘tsconfig.json‘, ‘tailwind.config.js‘, ‘postcss.config.js‘, ‘index.html‘, ‘package.json‘, ‘package-lock.json‘)

Test coverage (14 total)

QA verdict

PASS

Report 001 | Generated 2026-11-04 00:01:00 | reports/report-001.html